Department of Computer Engineering
University of Peradeniya
CO523 - Programming Languages

Lab 07: Subprograms, Parameter Passing, and Stack Frames

Objectives

- Understand subprograms (functions) and their role in program structure.
- Learn different parameter passing mechanisms.
- Understand how stack frames are created and destroyed during function calls.
- Simulate parameter passing using Python and C.
- Visualize call stack behavior through nested and recursive function calls.

1. Subprograms

A **subprogram** is a fundamental programming language construct that allows a program to be divided into smaller, manageable units. Each subprogram is designed to perform a specific task and can be executed independently by invoking it from another part of the program. Subprograms improve program readability, reduce code duplication, and support modular program design. In most modern programming languages, subprograms are implemented using **functions** or **procedures**.

When a subprogram is invoked, the program control transfers from the calling point to the subprogram body. After the subprogram completes execution, control returns to the point immediately following the call. This mechanism allows complex problems to be solved by breaking them down into simpler tasks. In Python and C, subprograms are implemented using **functions**, where Python functions always return a value (explicitly or implicitly), while C functions may or may not return a value.

E.g.;(Python)

```python
def add(a, b):
    c = a + b
    return c

result = add(4, 6)
print(result)
```

Result Explanation

- `add(4, 6)` creates a new stack frame.
- `Parameters a and b are stored in the stack frame.`
- `Local variable c is created inside the function.`

- After returning the value, the stack frame is destroyed.

E.g.; (C)

```c
#include <stdio.h>

int add(int a, int b) {
    int c =a+b;
    return c;
}

int main() {
    int result = add(4, 6);
    printf("%d", result);
    return 0;
}
```

In C, function calls also use stack frames, and parameters are passed using pass-by-value by default.

2. Parameter Passing Mechanisms

Parameter passing refers to the mechanism used to transfer data from a calling subprogram to a called subprogram. The method used determines whether modifications made inside the subprogram affect the original variables in the calling environment. Understanding parameter passing is essential for predicting program behavior and avoiding unintended side effects.

Common parameter passing mechanisms include:

- Pass-by-value
- Pass-by-reference
- Pass-by-object-reference(Python behavior)

Python and C primarily exhibit pass-by-value behavior, although reference-like behavior can be simulated using pointers in C and mutable objects in Python. Parameters are stored in the subprogram's stack frame and exist only during function execution.

2.1 Pass-by-Value

In **pass-by-value**, a copy of the actual argument is passed to the subprogram, so any changes made inside the function do not affect the original variable in the calling environment. This mechanism ensures data safety by keeping the caller's data unchanged.

In C, pass-by-value is the default parameter passing method. In Python, immutable data types such as integers, floats, and strings also show pass-by-value–like behavior. The copied parameter is stored in the function's stack frame and is destroyed when the function execution completes.

E.g.;(Python)

```python
def modify(x):
    x = x + 5
    print("Inside function:", x)

a = 10
modify(a)
print("Outside function:", a)
```

Result Explanation

- A copy of a is assigned to x
- Changes to x do not affect a
- Separate stack frames are used

E.g.;(C)

```c
#include <stdio.h>

void modify(int x) {
    x = x + 5;
    printf("Inside function: %d\n", x);
}

int main() {
    int a = 10;
    modify(a);
    printf("Outside function: %d\n", a);
    return 0;
}
```

Both Python (for immutable types) and C exhibit pass-by-value behavior in this case.

2.2 Pass-by-Reference Simulation

Pass-by-reference allows a subprogram to access and modify a variable in the calling environment by passing its reference or memory address instead of a copy. As a result, changes made inside the subprogram are visible outside the function.

In C, pass-by-reference is achieved using pointers, where the address of a variable is passed to the function. Python does not support true pass-by-reference, but similar behavior can be observed using mutable objects such as lists and dictionaries, whose contents can be modified within a function..

E.g.; Python (Using Mutable Objects)

```
def modify_list(lst):
        lst[0] = lst[0] + 5
        print("Inside function:", lst)

numbers = [10]
modify_list(numbers)
print("Outside function:", numbers)
```

Explanation

- The list reference is passed to the function
- Both caller and callee refer to the same object
- Changes persist outside the function

E.g.; (Using Pointers)

```
#include <stdio.h>

void modify(int *x) {
      *x = *x + 5;
}

int main() {
        int a = 10;
        modify(&a);
        printf("%d\n", a);
        return 0;
}
```

In C, pass-by-reference is achieved explicitly using **pointers**.

3. Stack Frames and Function Calls

A **stack frame** (or activation record) is a data structure that stores the information required for a subprogram's execution, including parameters, local variables, and the return address. Each function call creates a new stack frame that is pushed onto the call stack. The call stack follows a **Last In First Out (LIFO)** order, meaning the most recent function call completes first.

When a function finishes execution, its stack frame is removed, and control returns to the calling function. Understanding stack frames is essential for tracing execution flow and debugging errors.

E.g.;(Python)

```
def funcB(y):
    return y * 2

def funcA(x):
    return funcB(x + 1)

print(funcA(3))
```

Explanation

• funcA stack frame is created first
• funcB stack frame is created next
• funcB returns and its stack frame is removed
• funcA returns and its stack frame is removed

4. Stack Frames and Recursion

Recursion is a programming technique in which a subprogram calls itself to solve a problem by reducing it into smaller subproblems. Each recursive call creates a new stack frame with its own parameters and local variables. A recursive function must include a **base case** to terminate execution; otherwise, it results in infinite recursion and a stack overflow error.

Although recursion can offer clear and elegant solutions, it generally consumes more memory than iterative approaches due to the creation of multiple stack frames.

E.g.;(Python)

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

print(factorial(4))
```

Each recursive call adds a stack frame until the base case is reached. After that, stack frames are removed in reverse order.

5. Relationship Between Parameter Passing and Stack Frames

Parameter passing and stack frames are closely related, as parameters passed to a subprogram are stored within its stack frame. Whether a parameter is passed as a copy or as a reference determines how data is accessed and modified during execution. When the function returns, its stack frame is destroyed and all local variables and parameters cease to exist.

Understanding this relationship explains why some parameter changes persist after a function call while others do not, especially in nested and recursive subprogram executions.

Lab Tasks

Task 01

Write a program that:

- Accepts an integer input
- Passes the value to a function.
- Modifies the parameter inside the function.
- Prints values inside and outside the function.
- Demonstrates pass-by-value behavior.

(Implement in both Python and C)

Task 02

Write a program that:

- Uses a mutable object (Python list) / pointer (C).
- Modifies the value inside a function.
- Shows how the change affects the caller
- Clearly demonstrates pass-by-reference behavior

Task 03

Write a program with **two nested function calls** that:

- Passes parameters between functions
- Prints function entry and exit messages
- Simulates stack frame creation and destruction
- Draw the call stack at each stage

Task 04

Write a **recursive program** that:

- Calculates factorial or Fibonacci
- Prints the value of parameters at each call
- Identifies stack frame growth
- Explains what happens if the base case is removed

Task 05

Given the following function calls:

```
def f(a):
    b = a + 2
    return b

def g():
    x = 3
    y = f(x)
    return y
g():
```

• Manually simulate the stack frames
• Create a table showing:

- Function name
- Parameters
- Local variables
- Return values

(Repeat the same logic in C)

Submission

Create a Zip PDF file containing all your source codes and explanations. Name the file CO523_Lab07_EXXYYY.zip, where EXXYYY represents your E-number. Ensure all code is properly formatted and readable before submission.

Upload your answer sheet to the FEeLS by the given deadline.